

CO4 – AT3

Comparative Analysis

SOURCE CODE:

2. Shortest Path Comparison (Dijkstra vs Bellman-Ford)

```
#include <stdio.h>

#include <limits.h>

#define V 5

#define E 8

struct Edge {

    int src, dest, weight;

};

/* Dijkstra Algorithm */

void dijkstra(int graph[V][V], int source) {

    int dist[V], visited[V];

    int i, j, min, u;

    for (i = 0; i < V; i++) {

        dist[i] = INT_MAX;

        visited[i] = 0;

    }

    dist[source] = 0;

    for (i = 0; i < V - 1; i++) {
```

```

min = INT_MAX;

u = -1;

for (j = 0; j < V; j++) {

    if (!visited[j] && dist[j] < min) {

        min = dist[j];

        u = j;

    }

}

if (u == -1)

    break;

visited[u] = 1;

for (j = 0; j < V; j++) {

    if (graph[u][j] != 0 &&

        !visited[j] &&

        dist[u] != INT_MAX &&

        dist[u] + graph[u][j] < dist[j]) {

        dist[j] = dist[u] + graph[u][j];

    }

}

}

printf("\nDijkstra's Algorithm:\n");

for (i = 0; i < V; i++)

    printf("Source -> %d = %d\n", i, dist[i]);

```

```

}

/* Bellman-Ford Algorithm */

void bellmanFord(struct Edge edges[], int source) {

    int dist[V];

    int i, j;

    for (i = 0; i < V; i++)

        dist[i] = INT_MAX;

    dist[source] = 0;

    for (i = 1; i <= V - 1; i++) {

        for (j = 0; j < E; j++) {

            int u = edges[j].src;

            int v = edges[j].dest;

            int w = edges[j].weight;

            if (dist[u] != INT_MAX &&

                dist[u] + w < dist[v]) {

                    dist[v] = dist[u] + w;

                }

            }

        }

    }

    /* Check for negative cycle */

    for (j = 0; j < E; j++) {

        int u = edges[j].src;

```

```

int v = edges[j].dest;

int w = edges[j].weight;

if (dist[u] != INT_MAX &&
    dist[u] + w < dist[v]) {

    printf("\nNegative weight cycle detected!\n");

    return;

}

}

printf("\nBellman-Ford Algorithm:\n");

for (i = 0; i < V; i++)

    printf("Source -> %d = %d\n", i, dist[i]);

}

```

```

int main() {

```

```

    /*

```

Graph:

0 -> 1 = 6

0 -> 2 = 7

1 -> 2 = 8

1 -> 3 = 5

1 -> 4 = -4

2 -> 3 = -3

2 -> 4 = 9

3 -> 1 = -2

```
*/
```

```
int graph[V][V] = {
```

```
    {0, 6, 7, 0, 0},
```

```
    {0, 0, 8, 5, -4},
```

```
    {0, 0, 0, -3, 9},
```

```
    {0, -2, 0, 0, 0},
```

```
    {0, 0, 0, 0, 0}
```

```
};
```

```
struct Edge edges[E] = {
```

```
    {0, 1, 6},
```

```
    {0, 2, 7},
```

```
    {1, 2, 8},
```

```
    {1, 3, 5},
```

```
    {1, 4, -4},
```

```
    {2, 3, -3},
```

```
    {2, 4, 9},
```

```
    {3, 1, -2}
```

```
};
```

```
printf("Shortest Path Comparison\n");
```

```
printf("Starting Vertex = 0\n");
```

```
dijkstra(graph, 0);
```

```
bellmanFord(edges, 0);
```

```
return 0;
```

}